

Lista de Exercícios – Estrutura de Repetição (while)

Enunciado: Resolva os exercícios a seguir utilizando a linguagem Python, com foco na estrutura de repetição **while**. O objetivo é praticar o **controle de repetição**, incluindo **inicialização, condição e incremento**, além do uso de **contadores e acumuladores, validação de dados** e aplicação de **estruturas condicionais** dentro do loop. Utilize **operadores relacionais e lógicos** sempre que necessário, bem como a instrução **break** para interromper a repetição em situações específicas. Cada programa deve seguir a lógica de **entrada, processamento e saída**, sendo a **entrada** realizada por meio da função **input()**, o **processamento** feito com o uso do **while** para executar cálculos e verificações, e a **saída** apresentada de forma clara ao usuário. Procure manter o código **organizado**, com **boa identificação, nomes de variáveis claros e mensagens compreensíveis**. Procure desenvolver os programas de forma **organizada e legível**, utilizando **variáveis com nomes claros, identificação correta e mensagens informativas na saída dos dados**, facilitando a compreensão do funcionamento do algoritmo.

- 1)** Peça ao usuário um número inteiro positivo maior que 1. Utilizando while, verifique se esse número é **primo** (ou seja, divisível apenas por 1 e por ele mesmo). O algoritmo deve informar:
 - se o número é primo ou não
 - quantos divisores ele possui
- 2)** Peça ao usuário um número inteiro positivo. Utilizando while, percorra cada dígito desse número e calcule a soma total dos dígitos. Dica: use divisão inteira (//) e resto (%).
- 3)** Peça ao usuário um número inteiro positivo. Utilizando while, construa um novo número com os dígitos invertidos. Ao final, mostre:
 - número original
 - número invertido
- 4)** Peça ao usuário um número inteiro N (maior que 0). Utilizando while, exiba os **N primeiros termos da sequência de Fibonacci**. Além disso, mostre a soma dos termos gerados
- 5)** Peça ao usuário uma nota entre 0 e 10. Enquanto o valor informado for inválido (menor que 0 ou maior que 10), o algoritmo deve continuar solicitando uma nova entrada. Ao final, mostre a nota válida digitada.
- 6)** Peça ao usuário um valor inteiro positivo. Utilizando while, calcule a menor quantidade de notas necessárias para representar esse valor usando:
 - notas de 50
 - notas de 20
 - notas de 10
 - notas de 1

Mostre a quantidade de cada nota utilizada.

- 7)** Peça números inteiros ao usuário continuamente. O algoritmo deve parar apenas quando a soma dos números digitados ultrapassar 100. Ao final, mostre:
 - a soma total
 - a quantidade de números digitados
 - a média dos valores
- 8)** Peça ao usuário um número inteiro positivo. Utilizando while, verifique se o número é um palíndromo (ou seja, permanece igual quando invertido). Mostre:
 - número original
 - número invertido
 - resultado da verificação

9) Utilizando **while dentro de while**, exiba a tabuada de multiplicação do 1 ao 5. Cada número deve ter sua tabuada de 1 a 10. Organize a saída de forma legível.

10) [DESAFIO] Utilizando **while aninhado (um while dentro do outro)**, desenvolva um programa que leia 9 valores inteiros informados pelo usuário. A leitura deve ser feita de forma organizada, simulando **linhas e colunas**, ou seja, um laço externo será responsável por controlar as linhas e um laço interno por controlar as colunas.

Durante a execução, o programa deve:

- solicitar os valores ao usuário um a um;
- exibir os valores organizados em formato de grade (3 valores por linha);
calcular a soma de todos os números digitados;
- identificar o maior valor informado.

Ao final, o programa deve apresentar:

- os valores organizados em linhas e colunas;
- a soma total dos valores;
- o maior valor digitado.